

МИНОБРНАУКИ РОССИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«Пермский государственный национальный
исследовательский университет»

Институт компьютерных наук и
технологий

ОТЧЁТ
по индивидуальной работе №2
по дисциплине «Языки программирования»
Вариант 3 и 19

Работу выполнил
студент группы ИТ/О ИТ-2-2025 НБ 1
курса
Витухина А.М.
«14» июня 2026 г.

Работу проверил
Шилина А.В.
«20» июня 2026 г.

Пермь 2026

СОДЕРЖАНИЕ

1. Постановка задачи (3 вариант)	4
1.1 Текст задачи	4
1.2 Цель проекта	4
1.3 Функциональные требования	4
1.4 Технические требования	5
2. Алгоритм решения.....	6
2.1 Модульная структура программы	6
2.2 Алгоритм работы программы	6
2.3 Алгоритм генерации чисел Хэмминга	8
2.4 Алгоритм проверки данных.....	9
3. Тестирование.....	10
3.1 Тестирование на корректных данных	10
3.2 Проверка дополнительных функций	11
3.3 Тестирование на некорректных данных	11
3.4 Специфические сценарии алгоритма (Проверка логики очередей).....	12
4. Код программы	14
5. Постановка задачи (19 вариант).....	18
5.1 Текст задачи	18
5.2 Цель проекта	18
5.3 Функциональные требования	18
5.4 Технические требования	19
6. Алгоритм решения.....	19
6.1 Модульная структура программы	19
6.2 Алгоритм работы программы	20
6.3 Алгоритм проверки данных.....	21
7. Тестирование.....	22
7.1 Тестирование на корректных данных.....	22

7.2 Логические ошибки структуры (Конфликты).....	23
7.3 Ошибки файлового ввода	24
7.4 Ошибки ввода в меню	24
7.5 Нестандартные и стресс-сценарии	25
8. Код программы	28

1. Постановка задачи (3 вариант)

1.1 Текст задачи

Напечатать в порядке возрастания первых n натуральных чисел, в разложение которых на простые множители входят только числа 2, 3, 5. Идея решения: введем три очереди x_2 , x_3 , x_5 в которых будем хранить элементы, которые соответственно в 2, 3, 5 раз больше напечатанных, но еще не напечатаны. Рассмотрим наименьший из ненапечатанных элементов: пусть это x . Тогда он делится нацело на одно из чисел 2, 3, 5; x находится в одной из очередей и является в ней первым элементом (меньшие его уже напечатаны, а элементы очередей не напечатаны). Напечатав x , нужно изъять его из очереди и добавить в очередь кратные ему элементы. Длины очередей не превосходят числа напечатанных элементов. Изначально в очередях хранится по одному числу.

1.2 Цель проекта

Разработать программное обеспечение для генерации последовательности чисел Хэмминга (регулярных чисел) заданной длины в порядке возрастания. Программа должна демонстрировать эффективное использование динамических структур данных (очередей) для избежания сортировки всего массива кандидатов, а также предоставлять удобный пользовательский интерфейс с защитой от ошибок ввода.

1.3 Функциональные требования

Генерация последовательности:

1. Вычисление первых n чисел, простые множители которых принадлежат множеству $\{2, 3, 5\}$.
2. Строгий порядок возрастания результатов.
3. Отсутствие дубликатов в результирующей последовательности.

Пользовательский интерфейс:

1. Наличие циклического консольного меню.
2. Возможность запроса количества чисел (n).
3. Просмотр справки об алгоритме.

4. Корректный выход из программы.

Обработка данных:

1. Валидация ввода пользователя (проверка на целое число, положительность).
2. Обработка граничных случаев ($n = 0$, $n = 1$).
3. Предупреждение при запросе чрезмерно больших объемов данных.

1.4 Технические требования

1. Использовать динамические структуры данных (собственная реализация очереди на основе связного списка).
2. Запрещено использование специализированных библиотек коллекций.
3. Реализовать классы для узлов списка и очереди.
4. Соблюдать стандарт PEP 8.
5. Разделить код на логические модули (queue.py, logic.py, main.py).
6. Каждая функция должна выполнять одну логическую задачу.
7. Реализовать защиту от некорректного пользовательского ввода (буквы, отрицательные числа).

2. Алгоритм решения

2.1 Модульная структура программы

Программа разделена на три логических модуля:

1. **queue.py** - определение классов узла связного списка (Node) и очереди (Queue).
Реализует методы enqueue, dequeue, peek, is_empty.
2. **logic.py** - содержит функцию get_hamming_numbers(n), реализующую алгоритм генерации чисел с использованием трех очередей.
3. **main.py** - главный модуль, содержащий точку входа (main), циклическое меню пользователя и функции обработки ввода/вывода.

2.2 Алгоритм работы программы

НАЧАЛО

ПОВТОРЯТЬ

Вывести главное меню

Ввести пункт_меню

ЕСЛИ пункт_меню = 1 ТО

Запросить у пользователя количество чисел n

Попытаться преобразовать ввод в целое число

ЕСЛИ ввод некорректный (буквы, дробное число) ТО

Вывести сообщение об ошибке: "Введено некорректное значение"

ИНАЧЕ ЕСЛИ $n \leq 0$ ТО

Вывести сообщение об ошибке: "Количество чисел должно быть положительным"

ИНАЧЕ ЕСЛИ $n > 10000$ ТО

Запросить подтверждение у пользователя

ЕСЛИ пользователь отказался ТО

Отменить операцию

КОНЕЦ ЕСЛИ

ИНАЧЕ

Инициализировать три очереди q2, q3, q5

Добавить в результат число 1

Поместить в q2 значение 2
 Поместить в q3 значение 3
 Поместить в q5 значение 5
 ПОВТОРЯТЬ n-1 раз
 Найти минимальное значение min_val среди голов очередей q2, q3, q5
 Добавить min_val в результат
 ЕСЛИ голова q2 равна min_val ТО
 Удалить элемент из q2
 КОНЕЦ ЕСЛИ
 ЕСЛИ голова q3 равна min_val ТО
 Удалить элемент из q3
 КОНЕЦ ЕСЛИ
 ЕСЛИ голова q5 равна min_val ТО
 Удалить элемент из q5
 КОНЕЦ ЕСЛИ
 Добавить в q2 значение min_val * 2
 Добавить в q3 значение min_val * 3
 Добавить в q5 значение min_val * 5
 КОНЕЦ ПОВТОРА
 Вывести полученную последовательность (по 10 чисел в строке)
 КОНЕЦ ЕСЛИ
 ИНАЧЕ ЕСЛИ пункт_меню = 2 ТО
 Вывести справочную информацию об алгоритме и числах Хэмминга
 ИНАЧЕ ЕСЛИ пункт_меню = 3 ТО
 Вывести сообщение о завершении работы
 ВЫХОД ИЗ ЦИКЛА
 ИНАЧЕ
 Вывести сообщение об ошибке: "Неверный выбор"
 КОНЕЦ ЕСЛИ
 ПОКА пункт_меню ≠ 3
 КОНЕЦ

2.3 Алгоритм генерации чисел Хэмминга

НАЧАЛО

Входные данные: n - количество требуемых чисел

Выходные данные: список из n чисел в порядке возрастания

ЕСЛИ $n \leq 0$ ТО

 Вернуть пустой список

КОНЕЦ ЕСЛИ

Создать пустую очередь q_2

Создать пустую очередь q_3

Создать пустую очередь q_5

Добавить в результат число 1

Поместить в q_2 значение 2 ($1 * 2$)

Поместить в q_3 значение 3 ($1 * 3$)

Поместить в q_5 значение 5 ($1 * 5$)

ПОВТОРЯТЬ $n-1$ раз

$\min_val$ = минимальное из (голова q_2 , голова q_3 , голова q_5)

 Добавить $\min_val$ в результат

 // Удаление дубликатов: одно число может быть в нескольких очередях

 // Например, $6 = 2 * 3$ (в q_3) и $6 = 3 * 2$ (в q_2)

 ЕСЛИ голова $q_2 == \min_val$ ТО

 Извлечь элемент из q_2

 КОНЕЦ ЕСЛИ

 ЕСЛИ голова $q_3 == \min_val$ ТО

 Извлечь элемент из q_3

 КОНЕЦ ЕСЛИ

 ЕСЛИ голова $q_5 == \min_val$ ТО

 Извлечь элемент из q_5

```

        КОНЕЦ ЕСЛИ
        // Генерация новых кандидатов на основе найденного минимума
        Поместить в q2 значение min_val * 2
        Поместить в q3 значение min_val * 3
        Поместить в q5 значение min_val * 5
    КОНЕЦ ПОВТОРА

    Вернуть результат
    КОНЕЦ

```

2.4 Алгоритм проверки данных

1. **Тип данных:** количество чисел n должно быть целым числом.
2. **Диапазон значений:** n должно быть строго больше нуля ($n > 0$).
3. **Ограничение сверху:** при $n > 10000$ требуется дополнительное подтверждение пользователя для защиты консоли от переполнения выводом.
4. **Защита от нечислового ввода:** буквы, специальные символы, дробные числа должны вызывать понятное сообщение об ошибке, а не аварийное завершение программы.
5. **Корректность выбора пункта меню:** допустимы только значения 1, 2, 3; при вводе иных значений программа выводит предупреждение и возвращается к меню.
6. **Целостность очередей:** на каждом шаге алгоритма все три очереди должны содержать хотя бы по одному элементу (гарантируется инициализацией и тем, что на каждую извлечённую запись добавляется новая).

3.Тестирование

3.1 Тестирование на корректных данных

3.1.1 Минимально возможный ввод

Выберите пункт (1-3): 1

Введите количество чисел ($n > 0$): 1

Первые 1 чисел:

1

3.1.2 Стандартный малый набор

Введите количество чисел ($n > 0$): 5

Первые 5 чисел:

1 2 3 4 5

3.1.3 Проверка пропуска простых чисел (7, 11...)

Введите количество чисел ($n > 0$): 10

Первые 10 чисел:

1 2 3 4 5 6 8 9 10 12

3.1.4 Большой объем данных (проверка производительности)

Введите количество чисел ($n > 0$): 100

Первые 100 чисел:

1	2	3	4	5	6	8	9	10	12
15	16	18	20	24	25	27	30	32	36
40	45	48	50	54	60	64	72	75	80
81	90	96	100	108	120	125	128	135	144
150	160	162	180	192	200	216	225	240	243
250	256	270	288	300	320	324	360	375	384
400	405	432	450	480	486	500	512	540	576
600	625	640	648	675	720	729	750	768	800
810	864	900	960	972	1000	1024	1080	1125	1152
1200	1215	1250	1280	1296	1350	1440	1458	1500	1536

3.2 Проверка дополнительных функций

3.2.1 Просмотр справки

```
Выберите пункт (1-3): 2

--- Справка ---
Числа Хэмминга — это натуральные числа,
разложение которых на простые множители содержит только 2, 3 и 5.
Пример: 1, 2, 3, 4, 5, 6, 8, 9, 10, 12...

Алгоритм использует 3 очереди для хранения кандидатов,
что позволяет получать числа строго в порядке возрастания
без необходимости сортировки всего массива.
```

3.2.2 Корректный выход

```
Выберите пункт (1-3): 3
Завершение работы программы.

Process finished with exit code 0
```

3.3 Тестирование на некорректных данных

3.3.1 Ввод букв вместо числа

```
Введите количество чисел (n > 0): abc
Ошибка: Введено некорректное значение. Пожалуйста, введите целое число.
```

3.3.2 Ввод специальных символов

```
Введите количество чисел (n > 0): )(*&
Ошибка: Введено некорректное значение. Пожалуйста, введите целое число.
```

3.3.3 Ввод дробного числа

```
Введите количество чисел (n > 0): 3.14
Ошибка: Введено некорректное значение. Пожалуйста, введите целое число.
```

3.3.4 Пустой ввод (Enter)

```
Введите количество чисел (n > 0):
Ошибка: Введено некорректное значение. Пожалуйста, введите целое число.
```

3.3.5 Отрицательное число

Введите количество чисел ($n > 0$): -5
Ошибка: Количество чисел должно быть положительным.

3.3.6 Ноль

Введите количество чисел ($n > 0$): 0
Ошибка: Количество чисел должно быть положительным.

3.3.7 Несуществующий пункт меню

```
      ГЕНЕРАТОР ЧИСЕЛ ХЭММИНГА
      (Простые множители: 2, 3, 5)
=====
1. Получить последовательность
2. Справка об алгоритме
3. Выход
=====
Выберите пункт (1-3): 9
Неверный выбор. Попробуйте снова.
```

3.3.8 Буквы в выборе меню

```
=====
      ГЕНЕРАТОР ЧИСЕЛ ХЭММИНГА
      (Простые множители: 2, 3, 5)
=====
1. Получить последовательность
2. Справка об алгоритме
3. Выход
=====
Выберите пункт (1-3): q
Неверный выбор. Попробуйте снова.
```

3.4 Специфические сценарии алгоритма (Проверка логики очередей)

3.4.1 Проверка устранения дубликатов (число 30)

Введите количество чисел ($n > 0$): 16

Первые 16 чисел:

1	2	3	4	5	6	8	9	10	12
15	16	18	20	24	25				

3.4.2 Работа с большими степенями двойки

4.Код программы

Logic.py

```
from queue import Queue
def get_hamming_numbers(n):
    if n <= 0:
        return []

    # Инициализация очередей
    q2 = Queue()
    q3 = Queue()
    q5 = Queue()

    result = [1] # Первое число всегда 1

    # Заполняем очереди начальными значениями (1 * 2, 1 * 3, 1 *
5)
    q2.enqueue(2)
    q3.enqueue(3)
    q5.enqueue(5)

    # Генерируем оставшиеся n-1 чисел
    for _ in range(n - 1):
        # Находим минимальное значение среди голов очередей
        min_val = min(q2.peek(), q3.peek(), q5.peek())

        result.append(min_val)

        # Удаляем минимальное значение из всех очередей, где оно
есть.
        if q2.peek() == min_val:
            q2.dequeue()
        if q3.peek() == min_val:
            q3.dequeue()
        if q5.peek() == min_val:
            q5.dequeue()

        # Добавляем новые кандидаты в очереди
        q2.enqueue(min_val * 2)
        q3.enqueue(min_val * 3)
        q5.enqueue(min_val * 5)

    return result
```

main.py

```
import sys
from logic import get_hamming_numbers
```

```

def print_menu():
    """Вывод главного меню программы."""
    print("\n" + "=" * 40)
    print("    ГЕНЕРАТОР ЧИСЕЛ ХЭММИНГА")
    print("    (Простые множители: 2, 3, 5)")
    print("=" * 40)
    print("1. Получить последовательность")
    print("2. Справка об алгоритме")
    print("3. Выход")
    print("=" * 40)

def handle_sequence_generation():
    """Обработка пункта меню: генерация чисел."""
    try:
        user_input = input("\nВведите количество чисел (n > 0): ")
        n = int(user_input)

        if n <= 0:
            print("Ошибка: Количество чисел должно быть положительным.")
            return

        # Ограничение для безопасности консоли
        if n > 10000:
            confirm = input("Запрошено очень большое количество (>10000). Продолжить? (y/n): ")
            if confirm.lower() != 'y':
                print("Операция отменена.")
                return

        numbers = get_hamming_numbers(n)

        print(f"\nПервые {n} чисел:")
        # Вывод по 10 чисел в строке для удобства чтения
        for i, num in enumerate(numbers):
            print(f"{num:<12}", end="")
            if (i + 1) % 10 == 0:
                print()
        print() # Перенос строки в конце

    except ValueError:
        print("Ошибка: Введено некорректное значение. Пожалуйста, введите целое число.")
    except Exception as e:
        print(f"Произошла ошибка: {e}")

def show_help():
    """Вывод справки по алгоритму."""
    print("\n--- Справка ---")
    print("Числа Хэмминга — это натуральные числа,")

```

```

    print("разложение которых на простые множители содержит только
2, 3 и 5.")
    print("Пример: 1, 2, 3, 4, 5, 6, 8, 9, 10, 12...")
    print("\nАлгоритм использует 3 очереди для хранения
кандидатов,")
    print("что позволяет получать числа строго в порядке
возрастания")
    print("без необходимости сортировки всего массива.")

```

```

def main():
    """Основной цикл программы."""
    while True:
        print_menu()
        choice = input("Выберите пункт (1-3): ").strip()

        if choice == '1':
            handle_sequence_generation()
        elif choice == '2':
            show_help()
        elif choice == '3':
            print("Завершение работы программы.")
            break
        else:
            print("Неверный выбор. Попробуйте снова.")

```

```

if __name__ == "__main__":
    main()

```

queue.py

```

class Node:
    """Узел односвязного списка."""

    def __init__(self, data):
        self.data = data
        self.next = None

class Queue:

    def __init__(self):
        self.head = None
        self.tail = None
        self._size = 0

    def is_empty(self):
        """Проверка на пустоту."""
        return self.head is None

    def enqueue(self, item):
        """Добавление элемента в конец очереди."""

```

```

new_node = Node(item)
if self.tail is None:
    self.head = new_node
    self.tail = new_node
else:
    self.tail.next = new_node
    self.tail = new_node
self._size += 1

def dequeue(self):
    """Удаление и возврат первого элемента."""
    if self.is_empty():
        raise IndexError("Dequeue from empty queue")

    data = self.head.data
    self.head = self.head.next

    if self.head is None:
        self.tail = None

    self._size -= 1
    return data

def peek(self):
    """Возврат первого элемента без удаления."""
    if self.is_empty():
        raise IndexError("Peek from empty queue")
    return self.head.data

def size(self):
    """Текущий размер очереди."""
    return self._size

```

Ссылка на код: <https://github.com/nashua43/IKM/tree/main/вариант%203>

5. Постановка задачи (19 вариант)

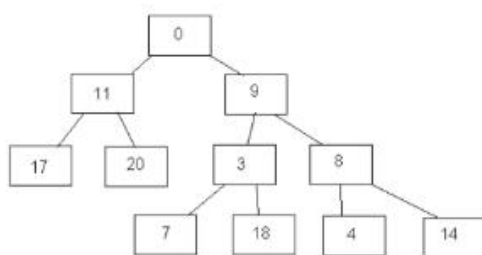
5.1 Текст задачи

- 19 В файле даны n целых чисел, и здесь же указан путь их размещения в бинарном дереве виде двоичного кода (коды не повторяются). Построить двоичное дерево целых чисел, в котором путь по дереву определяется указанным двоичным кодом в этом листе (1 – переход к правому потомку, 0 – переход к левому потомку). В корень автоматически заносится значение 0.

Например, для исходных данных:

15	111
18	101
3	10
8	11
9	1
11	0
7	100
4	110
17	00
20	01

должно быть построено такое дерево:



Учитывать ситуацию, когда дерево не может быть построено.

5.2 Цель проекта

Разработать программное обеспечение для считывания данных из файла, валидации путей размещения узлов и построения бинарного дерева согласно заданным двоичным кодам. Программа должна обеспечивать наглядную визуализацию построенной структуры в консольном режиме и предоставлять устойчивый интерфейс для работы с различными наборами входных данных.

5.3 Функциональные требования

Загрузка и обработка данных:

1. Чтение входных данных из текстового файла.

2. Парсинг пар «Значение — Двоичный код».
3. Валидация формата двоичных кодов (только символы '0' и '1').

Построение структуры:

1. Автоматическая инициализация корневого узла со значением 0.
2. Размещение узлов в дереве строго по заданному пути (0 — влево, 1 — вправо).
3. Детекция конфликтов:
 - Попытка занять уже существующий узел другим значением.
 - Попытка создать узел там, где уже существует внутренний узел с потомками.
4. Обработка ошибки невозможности построения дерева с информативным сообщением.

Интерфейс и вывод:

1. Циклическое меню пользователя.
2. Визуализация дерева в консоли (текстовое представление иерархии).
3. Возможность загрузки нового файла без перезапуска программы.

5.4 Технические требования

1. Использовать динамические структуры данных (собственная реализация узла бинарного дерева).
2. Запрещено использование специализированных библиотек для деревьев.
3. Реализовать классы для узлов (Node) и управления деревом (BinaryTree).
4. Соблюдать стандарт PEP 8.
5. Разделить код на логические модули (node.py, tree_builder.py, file_parser.py, main.py).
6. Каждая функция должна выполнять одну логическую задачу.
7. Реализовать защиту от некорректного пользовательского ввода и ошибок файловой системы.

6. Алгоритм решения

6.1 Модульная структура программы

Программа разделена на четыре логических модуля:

1. **node.py** — определение класса узла бинарного дерева (TreeNode), хранящего значение, ссылки на левого и правого потомков, а также информацию о том, является ли узел листом или внутренним узлом.
2. **file_parser.py** — модуль чтения и первичной валидации входного файла. Проверяет наличие файла, корректность формата строк и символов в двоичных кодах.
3. **tree_builder.py** — основная логика построения. Содержит класс TreeBuilder, который принимает распарсенные данные и последовательно размещает узлы в структуре, проверяя условия конфликта. Также содержит метод рекурсивной печати дерева.
4. **main.py** — главный модуль, содержащий точку входа (main), циклическое меню пользователя и обработку исключений высокого уровня.

6.2 Алгоритм работы программы

НАЧАЛО

Загрузить данные о размещении узлов из текстового файла

Если файл не найден или содержит ошибки

 Вывести сообщение об ошибке

 Запросить корректный путь к файлу

Конец если

Построить бинарное дерево по двоичным кодам путей

ПОВТОРЯТЬ

 Вывести главное меню

 Ввести пункт_меню

 ЕСЛИ пункт_меню = 1 ТО

 Загрузить новый файл с данными

 Если файл корректен

 Построить новое дерево

 Вывести сообщение об успехе

 Иначе

 Вывести подробное сообщение об ошибке

 Конец если

 ИНАЧЕ ЕСЛИ пункт_меню = 2 ТО

 Показать дерево в классическом виде (корень сверху)

```

ИНАЧЕ ЕСЛИ пункт_меню = 3 ТО
    Показать дерево в повёрнутом виде (корень слева)
ИНАЧЕ ЕСЛИ пункт_меню = 4 ТО
    Показать дерево по уровням (BFS-обход)
ИНАЧЕ ЕСЛИ пункт_меню = 5 ТО
    Выход из цикла
ИНАЧЕ
    Вывести сообщение об ошибке ввода
КОНЕЦ ЕСЛИ
ПОКА пункт_меню ≠ 5
КОНЕЦ

```

6.3 Алгоритм проверки данных

1. **Формат файла:** каждая строка данных должна содержать ровно 2 значения через пробел (целое число и двоичный код).
2. **Типы данных:** значение узла должно быть целым числом, код пути — строкой из символов '0' и '1'.
3. **Диапазоны:** код пути не может быть пустой строкой.
4. **Уникальность позиций:** в одну и ту же ячейку дерева нельзя записать два разных значения.
5. **Корневой узел:** корень автоматически инициализируется значением 0 и не требует указания в файле.
6. **Целостность путей:** если узел уже получил значение из файла, он может иметь потомков (промежуточные узлы создаются автоматически).
7. **Связность:** все указанные пути должны быть достижимы от корня через последовательность переходов '0' (влево) и '1' (вправо).

7. Тестирование

7.1 Тестирование на корректных данных

7.1.1 Пример из условия задачи (полный набор)

```
=====
ПОСТРОЕНИЕ БИНАРНОГО ДЕРЕВА
по двоичным кодам путей
=====
1. Загрузить файл и построить дерево
2. Показать текущее дерево
3. Выход
=====
Выберите пункт (1-3): 1
Введите имя файла: tree_valid.txt

Дерево успешно построено!
```

Выберите пункт (1-3): 2

```
--- Структура дерева ---
|
|   — 15
|   — 8
|   |   — 4
|   — 9
|   |   — 18
|   |   — 3
|   |   — 7
|   — 0
|   |   — 20
|   |   — 11
|   |   — 17
```

7.1.2 Минимальное дерево (один уровень)

```
Выберите пункт (1-3): 1
Введите имя файла: tree_minimal.txt
```

Дерево успешно построено!

```
=====
ПОСТРОЕНИЕ БИНАРНОГО ДЕРЕВА
по двоичным кодам путей
=====
1. Загрузить файл и построить дерево
2. Показать текущее дерево
3. Выход
=====
Выберите пункт (1-3): 2

--- Структура дерева ---
|
|   — 0
|   — 42
```

7.1.3 Линейная цепочка вправо

1	1 1
2	2 11
3	3 111

Введите имя файла: `tree_chain_right.txt`

Дерево успешно построено!

```
=====
ПОСТРОЕНИЕ БИНАРНОГО ДЕРЕВА
по двоичным кодам путей
=====
1. Загрузить файл и построить дерево
2. Показать текущее дерево
3. Выход
=====
Выберите пункт (1-3): 2
```

```
--- Структура дерева ---
|           --- 3
|           --- 2
|           --- 1
|           0
```

7.2 Логические ошибки структуры (Конфликты)

7.2.1 Конфликт значений (одна ячейка)

1	5 01
2	10 01

Выберите пункт (1-3): 1

Введите имя файла: `tree_conflict_structure.txt`

Ошибка построения: Конфликт: позиция '01' уже занята значением 5

7.2.2 Некорректный символ в коде

1	5 02
2	5 01

Выберите пункт (1-3): 1
Введите имя файла: *tree_invalid_chars.txt*

Ошибка файла: Строка 1: код '02' содержит недопустимые символы (разрешены только 0 и 1).

7.3 Ошибки файлового ввода

7.3.1 Файл не найден

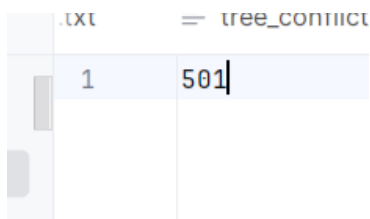
Выберите пункт (1-3): *nonexistent.txt*
Неверный выбор. Попробуйте снова.

7.3.2 Пустой файл

Выберите пункт (1-3): 1
Введите имя файла: *tree_empty.txt*

Ошибка файла: Файл пуст.

7.3.3 Неправильный формат строки



Выберите пункт (1-3): 1
Введите имя файла: *tree_bad_format.txt*

Ошибка файла: Строка 1: ожидается формат 'Значение Код' (через пробел).

7.3.4 Пустой путь к файлу

Выберите пункт (1-3): 1
Введите имя файла:

Ошибка файла: Файл '' не найден.

7.4 Ошибки ввода в меню

7.4.1 Буквы в выборе меню

```

=====
ПОСТРОЕНИЕ БИНАРНОГО ДЕРЕВА
по двоичным кодам путей
=====
1. Загрузить файл и построить дерево
2. Показать текущее дерево
3. Выход
=====
Выберите пункт (1-3): abc
Неверный выбор. Попробуйте снова.

```

7.4.2 Несуществующий пункт

```

-----
1. Загрузить файл и построить дерево
2. Показать текущее дерево
3. Выход
=====
Выберите пункт (1-3): 9
Неверный выбор. Попробуйте снова.

```

7.4.3 Отрицательное число

```

1. Загрузить файл и построить дерево
2. Показать текущее дерево
3. Выход
=====
Выберите пункт (1-3): -1
Неверный выбор. Попробуйте снова.

```

7.4.4 Дробное число

```

1. Загрузить файл и построить дерево
2. Показать текущее дерево
3. Выход
=====
Выберите пункт (1-3): 2.5
Неверный выбор. Попробуйте снова.

```

7.4.5 Попытка просмотра без загрузки

```

=====
Выберите пункт (1-3): 2

Дерево не построено или содержит ошибки. Загрузите файл (пункт 1).

=====

```

7.5 Нестандартные и стресс-сценарии

7.5.1 «Зеркальный» конфликт путей

1	5 0
2	10 1
3	20 00
4	30 11
5	40 0

Выберите пункт (1-3): 1

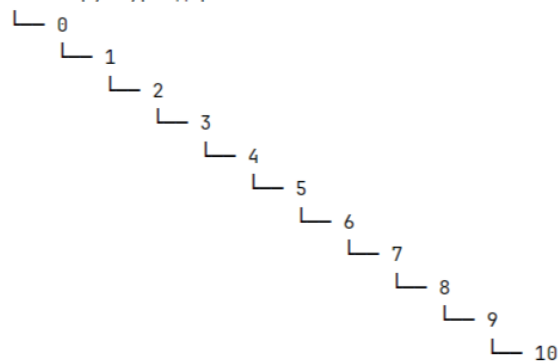
Введите имя файла: *tree_mirror_conflict.txt*

Ошибка построения: Конфликт: позиция '0' уже занята значением 5

7.5.2 Максимальная глубина для int32

1	1 0
2	2 00
3	3 000
4	4 0000
5	5 00000
6	6 000000
7	7 0000000
8	8 00000000
9	9 000000000
10	10 0000000000

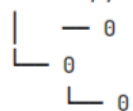
--- Структура дерева ---



7.5.3 Узел со значением 0

1	0 0
2	0 1

--- Структура дерева ---



7.5.4 Код пути длины 1 vs пустой код

1	5 0
2	10 1
3	15

Выберите пункт (1-3): 1

Введите имя файла: *tree_edge_paths.txt*

Ошибка файла: Строка 3: ожидается формат 'Значение Код' (через пробел).

8. Код программы

main1.py

```
from file_parser import FileParser
from tree_builder import TreeBuilder

def print_menu():
    print("\n" + "=" * 40)
    print("    ПОСТРОЕНИЕ БИНАРНОГО ДЕРЕВА")
    print("    по двоичным кодам путей")
    print("=" * 40)
    print("1. Загрузить файл и построить дерево")
    print("2. Показать текущее дерево")
    print("3. Выход")
    print("=" * 40)
```

tree_builder.py

```
from node import TreeNode
class TreeBuilder:

    def __init__(self):
        self.root = TreeNode(value=0)
        self.is_valid = True
        self.error_message = ""

    def insert(self, value, path_code):

        if not self.is_valid:
            return False

        current = self.root

        # Проходим по всем битам, кроме последнего
        for bit in path_code[:-1]:
            if bit == '0':
                if current.left is None:
                    current.left = TreeNode()
                    current = current.left
            elif bit == '1':
                if current.right is None:
                    current.right = TreeNode()
                    current = current.right
            else:
                self._set_error(f"Недопустимый символ в пути: '{bit}'")
                return False

        # Обработка последнего бита (целевая ячейка)
```

```

last_bit = path_code[-1]
if last_bit == '0':
    if current.left is None:
        current.left = TreeNode()
    target = current.left
elif last_bit == '1':
    if current.right is None:
        current.right = TreeNode()
    target = current.right
else:
    self._set_error(f"Недопустимый символ в пути: '{last_bit}'")
    return False

# Проверка конфликта: ячейка уже занята ДРУГИМ значением
if target.value is not None and target.value != value:
    self._set_error(f"Конфликт: позиция '{path_code}' уже занята значением {target.value}")
    return False

# Записываем значение (перезапись того же значения допустима)
target.value = value
return True

def _set_error(self, msg):
    self.is_valid = False
    self.error_message = msg

def print_tree(self, node=None, prefix="", is_left=True):
    """
    Рекурсивный вывод дерева в консоль (повёрнутое на 90°).
    """
    if node is None:
        node = self.root

    if node.right:
        new_prefix = prefix + ("|" if is_left else " ")
        self.print_tree(node.right, new_prefix, False)

    connector = "└─ " if is_left else "─ "
    val_str = str(node.value) if node.value is not None else ""

    print(prefix + connector + val_str)

    if node.left:
        new_prefix = prefix + (" " if is_left else "| ")
        self.print_tree(node.left, new_prefix, True)

```

file_parser.py

```

import os

class FileParser:
    @staticmethod
    def parse(filename):

        if not os.path.exists(filename):
            raise FileNotFoundError(f"Файл '{filename}' не найден.")

        entries = []
        with open(filename, 'r', encoding='utf-8') as f:
            lines = f.readlines()

        if not lines:
            raise ValueError("Файл пуст.")

        for line_num, line in enumerate(lines, 1):
            line = line.strip()
            if not line:
                continue

            parts = line.split()
            if len(parts) != 2:
                raise ValueError(f"Строка {line_num}: ожидается формат 'Значение Код' (через пробел).")

            val_str, code_str = parts

            # Валидация значения
            try:
                value = int(val_str)
            except ValueError:
                raise ValueError(f"Строка {line_num}: значение '{val_str}' не является целым числом.")

            # Валидация кода
            if not code_str:
                raise ValueError(f"Строка {line_num}: код пути пуст.")

            if not all(c in '01' for c in code_str):
                raise ValueError(
                    f"Строка {line_num}: код '{code_str}' содержит недопустимые символы (разрешены только 0 и 1).")

            entries.append((value, code_str))

        return entries

```

node.py

```
class TreeNode:

    def __init__(self, value=None):
        self.value = value
        self.left = None
        self.right = None

    def __str__(self):
        return str(self.value) if self.value is not None else "."
```

Ссылка на код : <https://github.com/nashua43/IKM/tree/main/вариант%2019>